gtfs-ingest: What Changed Since Dedupe and MiBici

From the within-feed dedupe and the MiBici stops CSV to routing-ready data — Guadalajara Metropolitan Area

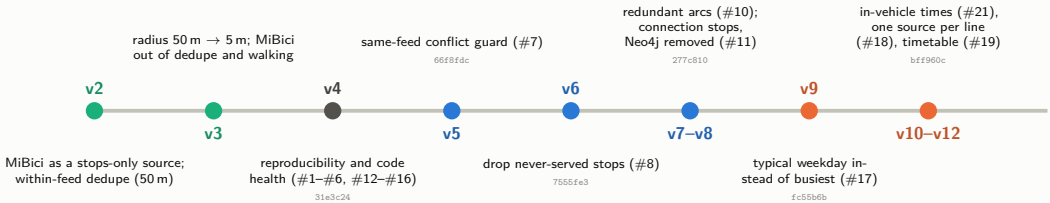
Víctor Rodríguez

November 1, 2026

Before: the run `out_mibici_v2` (October 2026, within-feed radius 50 m, MiBici with walk arcs) · after: `outputs/out_mibici_v12` · both with 2 GTFS feeds + MiBici, OSM network walk, 300 m proximity

From v2 to v12

Every step was measured on a full database run before moving on



8,877 → 10,203
physical stations

50 → 1
station-network components

0 → 3.29 M
timetable connections

Before and after

`out_mibici_v2` (or the first run that measured it) against `out_mibici_v12`

Measure	before	v12	Why
stop records in the outputs	14,349	12,768	1,581 never-served mdb stops left out (#8)
physical stations	8,877	**10,203**	opposite platforms no longer merge
within-feed alias pairs	2,697 (median 28.9 m)	247 ($\leq$ 4.6 m)	radius 50 m $\rightarrow$ 5 m
widest station	103 m	7.5 m	same-feed conflict guard (#7)
walk arcs touching a MiBici dock	5,756	0	docks are unlinked nodes
station-network weak components	50	**1**	never-served stops formed the pieces
one-way-reachable stops	86 (v3)	9	typical weekday, not a holiday (#17)
service dates	2020-01-01, 2024-01-01	2021-05-04, 2024-06-27	both old dates were holidays
routes in the routing network	274	249	38 lines published twice dropped (#18)
0-minute in-vehicle arcs	31 (v8)	1	aliases under one `stop_sequence` (#21)
`MT_L3` peak headway	230–330 s (v9)	420 s	directions relabelled (#21)
timetable	—	3,290,920 connections	`connections.csv.gz` (#19)
two runs, same files	no	**yes**	sorting, tie-breaks, manifest (#1–#5)

MiBici as a stops-only source

484 bike-share docks: geometry, no schedule

One predicate drives every branch · where the docks show up · out of dedupe and walking

One predicate drives every branch

`FeedSource.is_stops_only` — an explicitly named `.csv` is a feed that contributes stops only

- **Discovery** tags a named `.csv` as `kind="csv"`; a folder scan never picks up CSVs — a previous run's `near_stations.csv` would re-enter as a second network
- `feed_id` from the stem: `mibici.csv` → `mibici`
- The CSV reader resolves headers by alias (`Latitud`, `lat`, `LATITUDE`…), any delimiter, UTF-8 with BOM, UTF-16 or cp1252
- Phase 2 handles it **before** any service-date logic: no calendar, early return, status `stops_only`

Output	Docks?	How
stops, PostGIS, GeoPackage	✓	every stop output
proximity pairs	✓	`geographic_feed_ids`
`near_stations.csv`	✓	own row; neighbour with line –
station network	node	**no arcs** (unlinked)
dedupe	×	excluded from both rules
route graph, headways	×	no routes or trips
timetable	×	no trips

A bug dry runs could not show

The PostGIS proximity query filtered on *analysed* feeds only, so docks got no neighbours in full runs. Dry runs used a local join that ignored the list. Now stops-only feeds are in it.

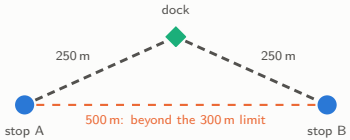
MiBici out of dedupe and out of walking

Two risks closed by rule, not by luck

DEDUPE

A dock named after a corner, within 30 m of a bus stop with the same name, could merge with it as "the same station". It never happened only because MiBici names start with `(GDL-001)`. **Now stops-only sources are excluded from both dedupe rules.**

WALKING



Docks were stepping stones between distant stops. **Now they have no walk arcs.**

0
docks merged with stops

5,756 → 0
walk arcs touching a dock

1,901 → 0
shortcuts that existed only via docks

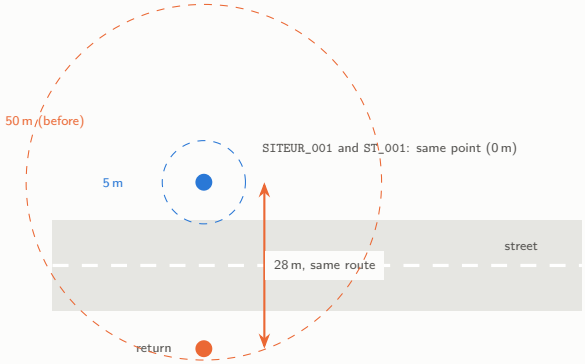
What counts as a station

From 8,877 to 10,203 physical stations

50 m → 5 m · the same-feed conflict guard (#7) · never-served stops (#8)

Why the within-feed radius went from 50 m to 5 m

A duplicate record sits at 0 m; the other direction's platform is across the street



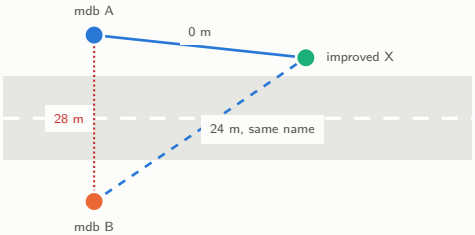
- At **50 m** the rule joined the outbound and the return platform: same route, 20–40 m apart.
- Two platforms became "one station", which gave a **free U-turn** when routing (0-minute alias arc).
- At **5 m** only records really listed twice are joined.

Measured

Within-feed pairs: 2,697 (median 28.9 m) → **247**, all between 0 and 4.6 m.

Chaining, and the same-feed conflict guard (#7)

Cross-feed pairs could still chain two opposite platforms of one feed through the other feed



Greedy union, closest pairs first: reject a pair that would put two stops of one feed more than 5 m apart in one group. Rejected pairs stay neighbours.

	v4	v5 (#7)
physical stations	10,533	11,054
groups with far same-feed stops ($>$ 15 m)	283	**0**
widest station	103 m	7.5 m
pairs rejected	—	1,077
route graph edges	9,112	9,112

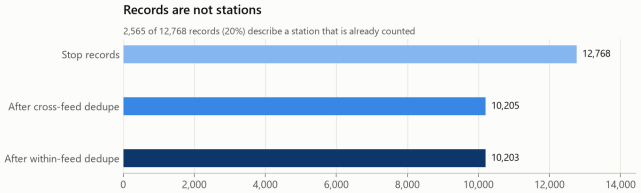
The rejected pairs' conflicts are 5.2 m or more apart (median 20.5 m): opposite platforms. A simulation before coding predicted exactly these numbers.

Never-served stops (#8)

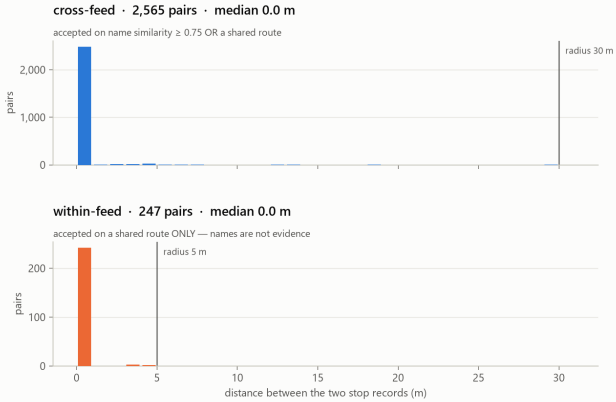
1,581 mdb stops are in `stops.txt` but in no `stop_times` row

- 738 duplicate a served stop of the other feed; 843 are listed with no trip at all.
- 111 of them formed the **49 small pieces** of the station network: nothing could reach them.
- Now left out of every output by default (`--no-drop-unserved-stops` keeps them); still loaded into PostGIS, which keeps the full inventory.

	v5 (keep)	v6 (drop)
stop records in outputs	14,349	12,768
physical stations	11,054	**10,203**
weak components (transit)	50	**1**
walk arcs	186,150	144,572
pairs rejected by #7	1,077	772
interchanges (multimodal)	2,708 (71)	2,565 (73)



Dedupe today



- **2,565 cross-feed pairs**, median 0.0 m: the feeds share a stop inventory.
- **247 within-feed pairs**, all $\leq$ 4.6 m: one station recorded twice.
- **1,604 groups**, widest 7.5 m, median span 0 m.
- MiBici: never merged.

A station is a platform group

Opposite platforms are separate stations joined by walk links — the right model for routing.

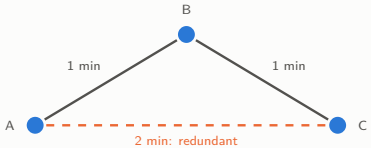
Network and outputs

Make the outputs cleaner and easier to use

Redundant arcs (#10) · where routes connect (#11) · Neo4j removed · a geography index

Redundant arcs (#10)

Many walk arcs never change a shortest time — reported always, pruned on request



Removed longest first, only if the remaining walking network still links the two stops within the same minutes; alias pairs keep a spanning tree per station.

	default	`--prune-station-network`
walk pairs	72,286	29,729
same_station pairs	2,812	2,565
weak components	1	1

Verified

Dijkstra from 300 random stops on both GraphMLs: identical `time_min` distances for all 3,683,106 reachable pairs.

Where routes connect (#11), and Neo4j removed

ROUTE CONNECTION CSVs

`route_connections.csv`	one row per route pair = the GraphML edges
`route_connection_stops.csv`	**where**: every route pair and stop pair behind an edge, with type and distance
`routes.csv`, `route_stops.csv`	decode the uids; `duplicate_of` since #18

For every route pair, the strongest type in `route_connection_stops.csv` equals the edge's type.

NEO4J EXPORT REMOVED

The CSVs replace the graph-database package: the same relations, readable in pandas, R or QGIS, with no server. The command, its module, tests and docs were deleted.

GEOGRAPHY INDEX

A GiST index on `geom::geography` for the proximity query:

2 min 18 s → 2 s

proximity query

The slow query had dropped a database connection mid-run.

Reproducibility

Two runs, the same files — and healthier code

Deterministic ids and order (#1–#3) · run manifest (#4) · golden test (#5)

ruff, packaging, smaller functions, faster ranges (#12–#16)

Two runs, the same files

Issue	Change
#1 IDs	`near_stations` `ID` numbered by `stop_uid`, not by `--feeds` order
#2 ties	every sort is stable (mergesort) with a full tie-break
#3 SQL	every query has `ORDER BY`; nearest-node ties broken by `osm_id`
#4 manifest	`run_manifest.json`: input sha256s, git commit, code hash, versions, parameters; no secrets
#5 golden	a fixed two-feed run byte-compared with committed CSVs

Checked on every full run since

v4 = v4b, v5 = v5b, v9 = v9b, v12 = v12b: every CSV, GraphML, GeoPackage layer and `connections.csv.gz` byte-identical; reports differ only in their timestamp.

An index trap

Adding `osm_id` straight to the KNN `ORDER BY` stopped PostGIS from using the spatial index: `conflate-osm` ran 23+ minutes instead of seconds. A 5-candidate subquery keeps the index and the tie-break.

Code health

#12 packaging	every subpackage in the wheel; CI checks it
#13 ruff	rules pinned, lint and format blocking in CI
#14 split	`_analyze_feed` and `run_analysis` split into small steps
#15 ranges	days with the same services analysed once: 3 min 02 s → 1 min 19 s
#16 cleanup	unused tooling and a stray venv removed

Formatting was AST-verified: no code or docstring changed.

303 → 347

tests, incl. live PostGIS

- CLI and config defaults share one constant, guarded by a test
- Sphinx `-W` and ruff in CI
- One default source per flag: `--fix-directions`, `--route-dedupe`, `--min-transfer-s`, ...

Routing-ready data

The inputs a routing benchmark needs

A typical weekday (#17) · in-vehicle times (#21) · one source per line (#18) · the timetable (#19)

A typical weekday, not the busiest day (#17)

"Busiest" was a holiday in both feeds: Sunday service stacked on weekday service

Rule: Mon–Fri dates with service that `calendar_dates` doesn't touch → the most common service set → the middle Tue–Thu date.

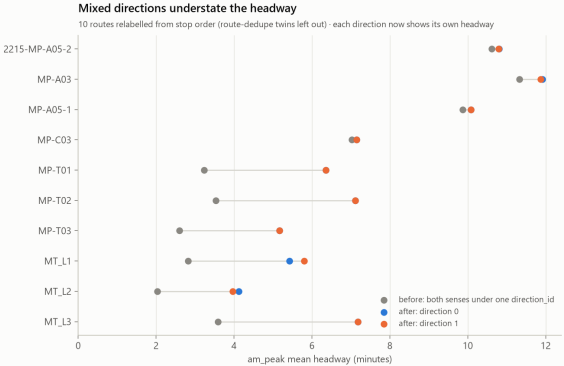
- `--date-rule busiest` keeps the old rule
- `--feed-service-date F=D` pins one feed
- `service_dates_note` explains why dates differ

	busiest (v8)	typical (v9)
improved date	2020-01-01	**2021-05-04**
mdb date	2024-01-01	**2024-06-27**
mdb trips	807	511
T16B directions	1	2
one-way-reachable stops	55	9
route graph routes	274	287

Still two years

The calendars never overlap (improved 2020–2022, mdb 2024), so each feed keeps its own day. After #18 only two improved routes remain in the routing inputs.

In-vehicle times (#21)



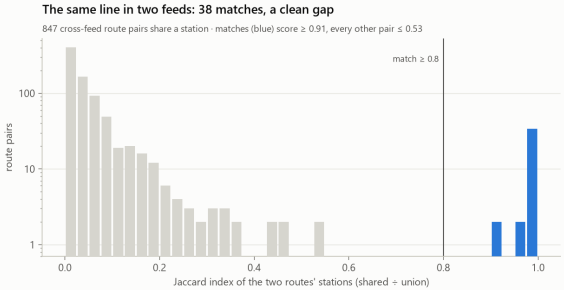
Directions: 19 trips on 19 routes carried both senses under one `direction_id`; relabelled from stop order. `MT_L3` now shows its published 420 s.

Stop order: mdb `MT_L1`/`MT_L2` list each station twice under one `stop_sequence`; one stop kept per sequence. 0-minute arcs 31 → 1.

Trust: `time_quality` on every arc and connection — 18 `too_fast` (> 80 km/h), 4 `too_slow` (T16B placeholders at 70 min), 1 `zero`. Times are kept.

Explained: the 484 stops with no travel time are the MiBici docks.

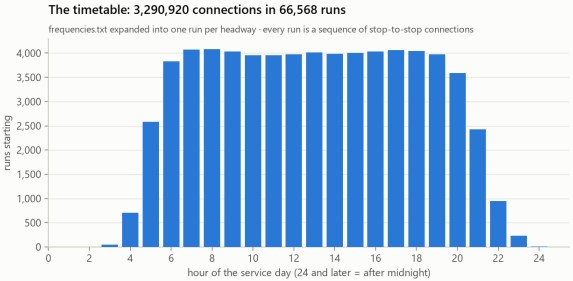
One source per line (#18)



- improved is almost a subset of mdb: 38 of its 40 routes are mdb routes with a 2215– prefix and identical departures
- Short names can't match lines: `L1`–`L3` are both Mi Tren and SiTren
- Stop sets mapped to stations: matches ≥ 0.91, others ≤ 0.53; the newer feed wins
- mdb lists every SiTren trip twice: 8 dropped

	before	after
routes in the graph	287	249
route links	10,255	7,902
`in_vehicle` arcs	26,388	24,036

The timetable (#19)



- `connections.csv.gz`: every vehicle hop of the day, frequencies expanded, sorted by departure
- `footpaths.csv`: walk links both ways, alias links and a 120 s change time per stop
- 61 MB compressed: 66,568 runs × ≈49 stops; about 550 MB as plain CSV
- Byte-identical across runs; every arc matches its connections
- A CSA scan from 08:00 reaches 99.8–99.9 % of the 10,499 served stops

Status

What the October review found, and where it stands

Open problems · review findings · next steps

The October review, revisited

	Problem (Oct 2026)	Now
FIXED	Opposite platforms still joined (283 groups)	conflict guard (#7): 0 groups, widest station 7.5 m
FIXED	Station network not connected (50 pieces)	never-served stops dropped (#8): 1 component
FIXED	86 stops reachable one way only	typical weekday (#17): 9, all route terminals
FIXED	Route graph "fragile" at T16B (2 bridges)	checked (#9): the bridges cut off two 3-stop feeders
FIXED	Many redundant walk arcs	counted on every run, `--prune-station-network` (#10)
BY DESIGN	Feeds analysed on different dates	typical weekday each; the calendars never overlap
BY DESIGN	Route graph depends on the 300 m radius	without walking the graph splits where BRT/rail is reached on foot
FIXED	CLI and config defaults differed	one constant per default, test-guarded
FIXED	Report omitted the within-feed radius	`dedupe_within_feed_distance_m` in the report
FIXED	Docks could merge with bus stops; docks extended walks	excluded from dedupe; unlinked in the network
FIXED	Free U-turns through alias arcs	5 m rule + conflict guard
FIXED	`ID` depended on `--feeds` order	numbered by `stop_uid` (#1)

Next steps

ROUTING DATA

- #20 frequency-based graph: waiting and transfer cost
- #22 access and egress: walk from any point to nearby stops
- #23 a clean merged GTFS for reference engines

ROUTING BENCHMARK

- #24 one interface for every algorithm
- #25 Dijkstra baselines: static, frequency-based, time-dependent
- #26 CSA and RAPTOR on `connections.csv.gz`
- #27 query sets, metrics and a `benchmark-routing` command
- #28 compare with OpenTripPlanner 2 or r5py

CODE HEALTH

- #29 figures regenerated from v12 — done for these decks
- #30 presentation decks and paper numbers
- #31 clean up old runs in `outputs/`

Ready now

A static graph for Dijkstra (`station_network.graphml`) and a full-day timetable for CSA and RAPTOR (`connections.csv.gz` + `footpaths.csv`), both reproducible from the same command.